applied sciences

MDPI

Article

In-Depth Analysis of Phishing Email Detection: Evaluating the Performance of Machine Learning and Deep Learning Models Across Multiple Datasets

Abeer Alhuzali [1,*], Ahad Alloqmani [1,†], Manar Aljabri [1,†] and Fatemah Alharbi [2]

[1] Department of Computer Science, Faculty of Computing and Information Technology, King Abdulaziz University, Jeddah 21589, Saudi Arabia; aalloqmani0005@stu.kau.edu.sa (A.A.); maljabri0083@stu.kau.edu.sa (M.A.)
[2] Computer Science Department, College of Computer Science and Engineering, Taibah University, Yanbu 46522, Saudi Arabia; fmhharbi@taibahu.edu.sa
* Correspondence: aalhathle@kau.edu.sa
† These authors contributed equally to this work.

Abstract: Phishing emails remain a primary vector for cyberattacks, necessitating advanced detection mechanisms. Existing studies often focus on limited datasets or a small number of models, lacking a comprehensive evaluation approach. This study develops a novel framework for implementing and testing phishing email detection models to address this gap. A total of fourteen machine learning (ML) and deep learning (DL) models are evaluated across ten datasets, including nine publicly available datasets and a merged dataset created for this study. The evaluation is conducted using multiple performance metrics to ensure a comprehensive comparison. Experimental results demonstrate that DL models consistently outperform their ML counterparts in both accuracy and robustness. Notably, transformer-based models BERT and RoBERTa achieve the highest detection accuracies of 98.99% and 99.08%, respectively, on the balanced merged dataset, outperforming traditional ML approaches by an average margin of 4.7%. These findings highlight the superiority of DL in phishing detection and emphasize the potential of AI-driven solutions in strengthening email security systems. This study provides a benchmark for future research and sets the stage for advancements in cybersecurity innovation.

Keywords: phishing email detection; cybersecurity; artificial intelligence (AI); deep learning (DL); machine learning (ML); spam filtering; threat detection; transformer models

check for updates

Academic Editor: Andrea Prati

Received: 27 February 2025
Revised: 16 March 2025
Accepted: 17 March 2025
Published: 20 March 2025

Citation: Alhuzali, A.; Alloqmani, A.; Aljabri, M.; Alharbi, F. In-Depth Analysis of Phishing Email Detection: Evaluating the Performance of Machine Learning and Deep Learning Models Across Multiple Datasets. *Appl. Sci.* **2025**, *15*, 3396. https://doi.org/10.3390/app15063396

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Phishing email attacks remain among the most prevalent and effective cyberattack vectors, exploiting human trust to gain unauthorized access to sensitive information. These attacks typically involve deceptive emails that appear to originate from legitimate sources [1], persuading victims to share credentials [2], transfer funds [3], or perform other harmful actions [4]. The increasing sophistication of phishing techniques, particularly using artificial intelligence (AI) to craft highly realistic phishing emails [5], has significantly weakened traditional detection mechanisms. Conventional approaches, such as feature-based filtering that relies on URLs, IP addresses, and sender information [6,7], struggle to identify AI-generated phishing emails due to their ability to mimic legitimate communication with unprecedented accuracy.

As cybercriminals leverage AI to enhance deception and evade detection, developing more advanced and adaptive security solutions is imperative to detect these evolving

threats in real time [8]. Machine learning (ML) and deep learning (DL) have been extensively studied in the literature as optimal methods for the automated detection of phishing emails. ML techniques, such as naive Bayes [9], logistic regression [10], and random forest [11], have been widely utilized for their ability to classify emails based on handcrafted features like sender information, content structure, and metadata [12–17]. These methods offer a relatively straightforward and computationally efficient approach to detecting phishing emails. On the other hand, DL models, including convolutional neural networks (CNNs) [18], recurrent neural networks (RNNs) [19], long short-term memory (LSTM) [20], and transformer-based architectures like BERT [21], are increasingly recognized for their ability to automatically extract complex features from raw email data [22,23]. By leveraging advanced neural architectures, DL models can analyze email content's contextual and semantic nuances, enabling higher accuracy and robustness in phishing detection. The success of ML and DL models in other domains [24,25] has fueled their application in phishing detection, establishing them as essential tools for enhancing email security.

Despite extensive research on phishing detection, most studies are limited in scope, often focusing on a few datasets or a narrow range of models [12,26]. This narrow focus restricts their applicability in real-world scenarios [27,28], where phishing patterns vary significantly across datasets. Furthermore, DL models, which have shown exceptional promise in various domains, remain underexplored in phishing email detection [29]. Addressing these gaps is critical for developing robust and adaptive solutions to combat evolving phishing tactics effectively.

Research Gap. While significant research has been conducted on phishing detection, several gaps remain:

- Most prior studies rely on a single or a limited number of datasets, which restricts the generalizability of their findings to diverse phishing scenarios.
- Despite the success of DL in other domains, its potential in phishing detection—especially using advanced architectures such as transformer-based models—has not been fully investigated.
- Existing studies often lack a standardized framework for consistently implementing and evaluating ML and DL models across multiple datasets.

Research Questions. This study addresses the following research questions:

RQ1: What are the most effective ML and DL models for phishing email detection?
RQ2: How does the performance of ML and DL models vary when applied to multiple individual datasets versus a merged dataset?
RQ3: What are the key performance differences between ML and DL models in terms of accuracy, robustness, and generalizability for phishing email detection?
RQ4: Which DL models demonstrate superiority over ML models in phishing detection, and why?

Key Contributions. To answer these questions, this study presents the following contributions:

1. We develop a novel framework for implementing and evaluating 14 ML and DL models for phishing email detection, ensuring a standardized and comprehensive assessment of their effectiveness.
2. The study utilizes nine publicly available datasets and a merged dataset, addressing the limitations of previous research that relied on single or limited datasets, thereby enhancing the robustness and generalizability of the findings.
3. Through an extensive comparison using various evaluation metrics (e.g., accuracy, precision, recall, F1-score), we demonstrate that deep learning models, particularly

transformer-based architectures like BERT [21] and RoBERTa [30], consistently outperform traditional ML models, reinforcing the potential of AI-driven phishing detection.

The remainder of this paper is organized as follows: Section 2 provides an overview of related work. Sections 3 and 4 describe the methodology employed and discuss the experimental results. Finally, Section 5 concludes the study and highlights future research directions.

2. Related Work

Detecting phishing emails has been a significant focus of research within the cybersecurity domain, driven by these attacks' increasing sophistication and prevalence. Numerous studies have investigated using ML and DL approaches to enhance the detection and classification of phishing emails. In this section, we review key studies that leverage these techniques to address the challenges posed by phishing emails. The key findings from the literature are summarized in Table 1.

Table 1. Summary of Related Work on ML and DL Techniques for Phishing Email Detection

Ref.	Year	Dataset	Model (s)	Result (Accuracy)
[31]	2025	Public datasets (UCI, SpamAssassin)	CNN, XGBoost, RNN, SVM, BERT-LSTM	99.55%
[32]	2024	Enron and multiple phishing repositories	Logistic Regression, Random Forest, SVM, Naive Bayes, distilBERT, BERT, XLNet, RoBERTa, ALBERT	99.43%
[12]	2024	Merged (6 datasets)	SVM, Naive Bayes, Random Forest	99.19%
[22]	2024	Kaggle phishing dataset	DistilBERT	98.48%
[33]	2023	Public datasets (UCI, Kaggle)	CNN, RNN, LSTM, BERT	99.61%
[26]	2023	Nazario phishing corpus	ANN, RNN, CNN	99.51%
[13]	2022	SpamAssassin	LSTM, Gaussian Naive Bayes	97.18%
[23]	2022	N/A	BERT, DistilBERT	99%
[14]	2021	Custom dataset	CNN, LSTM	96.34%
[34]	2021	Fraud dataset	GCN	98%
[15]	2020	SMS Spam Collection v.1	SVM, Decision Tree, Random Forest	98.77%
[35]	2019	IWSPA-AP 2018	RCNN	99.848%

Chinta et al. [31] highlighted the growing cyber threat of phishing attacks and explored machine learning-based phishing email detection. The phishing email dataset used in their paper is a curated dataset of public datasets such as SpamAssassin and datasets from the UCI Machine Learning Library. Several models are trained to detect phishing emails, including CNN, XGBoost, RNN, SVM, and a BERT-LSTM hybrid model. Based on the performance evaluation, the BERT-LSTM model has the best results, with an accuracy of 99.55% and an F1-score of 99.24%. In [32], the authors compared traditional ML models (i.e., logistic regression, random forest, SVM, and naive Bayes) with transformer models (i.e., distilBERT, BERT, XLNet, RoBERTa, and ALBERT) for phishing email detection using a dataset of 119,148 emails from the Enron dataset and multiple phishing repositories. Results show that RoBERTa achieved the highest accuracy 99.43%, while SVM outperformed traditional ML models with 98.76% accuracy. Transformer-based models reduced misclassifications by 53.7% but struggled with mixed-language content.

In [12], the authors developed a web-based AI platform for detecting phishing emails, employing ML models such as Support Vector Machine (SVM), multinomial naive Bayes,

and random forest. These models used Term Frequency-Inverse Document Frequency (TF-IDF) for feature extraction. Their merged dataset, combining six sources, contained approximately 82,500 emails, with SVM achieving the highest accuracy of 99.19%

Recent work [22] proposed a transformer-based approach using a fine-tuned DistilBERT model with explainable AI (XAI) techniques. A phishing dataset from Kaggle was balanced using resampling techniques, resulting in a testing accuracy of 97.50% on the imbalanced dataset and 98.48% on the balanced dataset. However, this study did not compare the model to other state-of-the-art methods on the same dataset.

Similarly, DL techniques such as CNNs, RNNs, LSTMs, and BERT were employed in [33] to identify phishing emails. Publicly available datasets were obtained from the UCI ML Repository, Kaggle, and Google Dataset Search, with BERT achieving an accuracy of 99.61%.

In [26], the authors proposed a DL detection model using artificial neural networks (ANNs), RNNs, and CNNs, leveraging a real-world phishing corpus from Nazario. Their approach addressed data imbalance effectively, achieving 99.51% accuracy.

Other studies explored different approaches, such as [13], which compared Gaussian naive Bayes and LSTM models on the SpamAssassin dataset. LSTM achieved a significantly higher accuracy of 97.18% compared to naive Bayes. Similarly, ref. [23] applied BERT and DistilBERT pre-trained transformer models to address traditional feature extraction limitations, achieving 99% accuracy.

In [14], CNN and LSTM models were evaluated on a custom dataset of phishing and legitimate emails. CNN with word embeddings achieved 96.34% accuracy, while Naive Bayes demonstrated the fastest computation time with 75.72% accuracy.

A graph convolutional network (GCN) model for phishing detection was proposed in [34], trained on a publicly available fraud dataset, and achieved an accuracy of 98%. Similarly, the authors in [15] employed various ML classifiers on a spam dataset, with SVM achieving the highest accuracy of 98.77%.

Finally, the authors in [35] introduced THEMIS, a recurrent convolutional neural network (RCNN) model utilizing multilevel vectors and attention mechanisms. THEMIS demonstrated excellent performance, achieving 99.848% accuracy on an anti-phishing shared task dataset.

While significant research has been conducted on phishing email detection, key gaps persist: (1) many studies use a single or limited number datasets, restricting generalizability; (2) advanced architectures, such as transformer-based models, are insufficiently examined for phishing email detection; and (3) lack of a unified framework for consistent model evaluation across datasets. To bridge these gaps, our work introduces a comprehensive evaluation framework incorporating diverse datasets and advanced ML and DL models to enhance phishing detection accuracy, reproducibility, and generalizability.

3. Research Methodology

To address our research questions, we propose a structured framework for implementing and assessing 14 well-known base ML and DL models. To address the discussed research gaps, this framework integrates several state-of-the-art algorithms frequently used in the literature for phishing email classification, including XGBoost and BERT. The evaluation is conducted on nine widely recognized public datasets, ensuring a diverse and representative assessment of model performance. In addition, we have created a curated dataset that integrates all nine datasets to further analyze model effectiveness under a unified dataset. Figure 1 represents the overall structure of our framework. Each component in the framework is explained below.

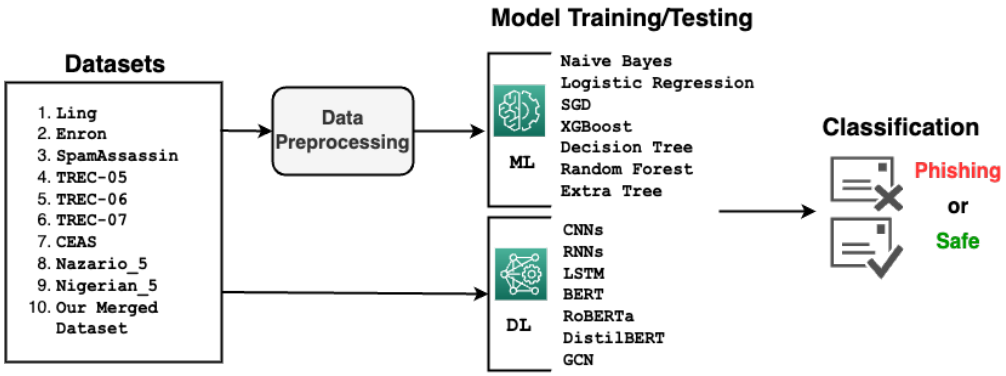


Figure 1. Overview of the Proposed Framework.

3.1. Data Collection

We utilize nine email datasets commonly used in phishing and spam detection research to answer RQ2 (models' effectiveness on multiple individual datasets and a merged dataset). In addition, we have created a merged dataset of all nine public datasets. The following list summarizes the datasets used in our framework.

- Ling: Derived from the Linguist List online email communication in the early 2000s [36], it consists of 458 spam and 2401 legitimate emails. This dataset is useful for linguistics-related studies but is limited in scope and size.
- Enron: The dataset was extracted from the Enron email corpus around the year 2005 [37]. It consists of 13,976 spam and 15,791 ham emails. It includes extensive corporate communications but with limited diversity.
- SpamAssassin: Developed by the Apache Software Foundation from 2002 to 2006 [38], it consists of 1670 spam and 3928 ham emails. It provides diverse public spam emails but might contain outdated tactics.
- TREC: It includes TREC-05 (2005), TREC-06 (2006), and TREC-07 (2007), focusing on diverse email communications [39,40]. TREC-05 consists of 20,643 spam and 31,842 ham emails. Its diverse sources enhance utility but may include specific period biases. TREC-06 consists of 3642 spam and 12,079 ham emails. It is useful for text classification but may not mirror real-world interactions. Dataset TREC-07 consists of 29,093 spam and 24,338 ham emails. It is a large and diverse dataset, but its periodic nature may affect consistency.
- CEAS: This dataset is from the CEAS 2008 Challenge Lab Evaluation Corpus [41]. It contains 21,839 spam and 16,853 ham emails. It captures a broad range of communications but may reflect dated spam characteristics.
- Nazario_5: It is a balanced dataset of different sources [42]: 1469 phishing and 1482 legitimate emails. It is useful for model testing; however, it lacks broad phishing tactic coverage.
- Nigerian_5: This is a publicly available dataset featuring advance-fee scams from 1998 to 2007 [43]. It contains 1586 scam emails and 2967 legitimate emails.
- Our merged dataset: We integrated all nine datasets, covering various features and fields. This merged dataset contains 206,057 emails, of which 94,376 (45.81%) are phishing emails, and 111,681 (54.19%) are safe emails. The spam/scam emails from all datasets were labeled as phishing emails. We have used the `Pandas library` in Python to combine the data into a single DataFrame. To balance this dataset, we used the down-sampling technique to reduce the number of safe emails to match the number of phishing emails.

3.2. Data Preprocessing

For ML models, we first remove any null values and drop duplicates. We retain only the `Email Text` and `Email Type` columns, renaming them for clarity. For label encoding, we converted `Email Type` labels into a numerical form: `Safe Email` as 0 and `Phishing Email` as 1. Next, we eliminated hyperlinks, punctuation, and extra spaces from the text. Finally, we converted the text into vectors using TF-IDF Vectorizer, chosen for its superior performance as noted in [12]. To train the DL models, we opted not to preprocess the text, aligning with the approach recommended by [13]. Their study demonstrates that DL models, particularly LSTMs, achieve superior performance—higher accuracy, recall, precision, and F1 score—when trained on raw email text without preprocessing.

3.3. Models Selection

Our goal in this work is not to improve machine learning (ML) and deep learning (DL) models but to develop a comprehensive evaluation framework. This framework provides an in-depth assessment of the accuracy and performance of different base ML and DL models across a wide range of datasets. To answer RQ1 (best classification algorithm), we selected well-known base classification algorithms used in recent phishing email detection research, as discussed in Section 2. We classify the algorithms into two main categories: ML and DL models. A brief description of these algorithms is provided in the following subsections.

3.3.1. Selected Machine Learning Models

Naive Bayes is a type of probabilistic classifier developed from Bayes' theorem and assumes feature independence, often referred to as the "naive" assumption [44]. It also assumes that all features in the dataset contribute equally to the outcome. These assumptions simplify its computation by requiring only a single probability per variable. While these assumptions are often unrealistic, the algorithm remains effective, especially with small datasets and for problems such as phishing email detection [12–15]. The model classifies emails as either phishing or legitimate given the extracted features by computing the posterior probability of each class.

Logistic Regression is a commonly employed statistical technique for binary classification tasks [45] such as phishing email detection [15,46]. It estimates the probability of an email being phishing or legitimate by fitting data to a logistic function. This function models the relationship between a dependent binary variable and one or more independent variables. For phishing detection, the model analyzes various email features to determine the likelihood that an email is malicious. An example of an email feature is email textual content (i.e., presence of suspicious words, grammatical errors, urgent language, request for sensitive information, etc.). Each feature has a weight assigned by the logistic regression model, which collectively contributes to the final probability score. If this probability score exceeds a certain threshold, the email is classified as phishing; otherwise, it is considered safe.

Stochastic Gradient Descent Classifier (SGD) operates by randomly selecting a subset of samples from the dataset for each iteration, rather than using the entire dataset [47]. This method results in different outputs for each iteration because the output reflects a partial derivative based on the subset of parameters used from the input. Specifically, email text is converted into numerical features. The classifier is trained using labeled email data: phishing and safe email. During each iteration, SGD updates model parameters based on a randomly selected subset of samples. It adjusts the weights to minimize classification error and uses a loss function to guide the updating of the weights. After the training, the model predicts if an incoming email is phishing or safe based on the learned feature weights.

EXtreme Gradient Boosting Classifier (XGBoost) represents an advanced version of gradient boosting algorithms [48]. It is designed for scalable and efficient tree boosting. It uses an ensemble learning method by combining multiple decision trees to enhance predictive accuracy. Each tree in the sequence is built to specifically address and correct the errors of its predecessors through gradient boosting. XGBoost is effective for structured data classification tasks, including phishing email detection [33,46,49]. In our work, we utilized the base model, where email content is converted into numerical features, and the model was trained. After training, XGBoost assigns a probability score for classifying an email as phishing or safe.

Decision Tree is a ML classifier used for classification and regression tasks [48]. It includes a recursive partitioning that assesses attributes or features based on specific purity metrics. The model creates multiple decision nodes, each representing a feature that effectively recognizes the classes based on the training data. At each node, the tree branches into outcomes or subsequent decisions. This branching continues until the model can classify an email as phishing or legitimate by following the path of decisions made from the root to a leaf of the tree [50]. This model is used in many phishing detection works such as [15].

Random Forest is a supervised ML algorithm used for classification and regression tasks [51]. It constructs multiple decision trees from the training dataset. Then it determines the final and best prediction of an email through an ensemble voting mechanism (for classification) or averaging (for regression). Random Forest executes several trees in parallel, which reduces overfitting and improves model accuracy [52]. This method has a broad array of applications due to its robust performance and has been used in many phishing detection papers such as [12,15].

Extra Tree ensemble is a decision-tree-based ensemble method, closely related to Random Forest [53]. It utilizes tree-based supervised models to assess feature relevance for making predictions. Unlike Random Forest, which uses bootstrap samples, Extra Tree algorithm fits each decision tree to the entire dataset and chooses splits randomly. While Random Forest selects the optimal split at each node, Extra Tree makes this selection randomly. This model was used to detect phishing emails in research works as in [54].

3.3.2. Selected Deep Learning Models

Convolutional neural networks (CNNs) are the best-known and widely adopted DL models. The key advantage of CNNs over previous architectures is that they automatically recognize the important features without human supervision. It has been demonstrated that CNNs, which were first developed for computer vision, perform well on text classification tasks [55–59] in addition to other traditional natural language processing (NLP) tasks [60,61]. Our framework uses the following CNN model architecture: an input layer consists of TF-IDF vectors of the email text, followed by a fully connected dense layer with 128 neurons and ReLU activation. A dropout layer with a rate of 0.3 is applied to prevent overfitting. Next, another dense layer with 64 neurons and ReLU activation is included, followed by another dropout layer with a rate of 0.3. Finally, the output layer comprises a single neuron with sigmoid activation for binary email text classification.

Recurrent neural networks (RNNs) are mostly used in speech processing and NLP contexts. Unlike CNNs, an RNN employs sequential data in its network. This feature is vital for a wide range of applications since the embedded structure in the data sequence delivers meaningful information. For example, understanding the context of a sentence is essential for determining the meaning of a specific word in it. Thus, an RNN can be viewed as a short-term memory unit, with x representing the input layer, y representing the output layer, and s representing the state (hidden) layer [62]. We have used the following RNN model architecture: an embedding layer that converts tokenized sequences into

dense vector representations. It has an input dimension equal to the vocabulary size, an output dimension of 128 (vector size for each word), and an input length of 200 (maximum sequence length). The SimpleRNN layer has 128 units to capture sequential patterns in the text and uses the ReLU activation function. The dense layer is fully connected with 64 neurons and uses ReLU as the activation function. The output layer is a fully connected layer with a single neuron and a sigmoid activation function for binary classification.

Long short-term memory (LSTM) denotes that the LSTM network can produce long-term or short-term delays for a variety of operations [63]. An LSTM cell consists of four components: the cell state, the input gate, the forget gate, and the output gate. LSTM employs these four components to ensure long-term and short-term reliance on successive data. Compared to traditional RNNs, LSTMs have the benefit of processing long sequences more efficiently than RNNs because of their strong internal structure [64]. The LSTM model architecture used in our framework includes an embedding layer that converts tokenized sequences into dense vector representations. It has an input dimension equal to the vocabulary size, an output dimension of 128 (embedding size for each word), and an input length of 200 (maximum sequence length). The LSTM layer includes 128 units to capture long-term dependencies in the email text. It uses the Tanh activation function and does not return sequences; it outputs the last hidden state. The dense layer is a fully connected layer with 64 neurons and uses the ReLU activation function. Finally, the output layer is a fully connected layer with a single neuron. It uses the sigmoid activation function to produce probabilities for email text classification.

Bidirectional encoder representations from transformers (BERT) is a widely used model developed by Google for NLP tasks [21]. Unlike traditional transformers that use both encoders and decoders, BERT utilizes only the encoder component.BERT processes input data using a multi-layer transformer encoder. It first embeds input tokens with a token embedding layer, then applies multiple self-attention layers within the encoder [65]. This allows BERT to understand the context by reading text bi-directionally, considering both words' left and right contexts. For specific tasks, the model undergoes fine-tuning on relevant datasets. Upon receiving text, BERT outputs category probabilities, selecting the most likely classification based on the highest probability. Many research works have recently used BERT to detect phishing emails [23,33], fake news, and spam [66]. In our work, we have utilized the base BERT model, which has the following architecture: a BERT encoder that utilizes pre-trained BERT Transformer layers to encode input sequences into dense vector representations. It captures contextual and semantic relationships within the text, producing 768-dimensional embeddings for each input sequence. A dense layer with 64 neurons processes these embeddings using the ReLU activation function. Finally, the output layer consists of a single neuron with a sigmoid activation function designed for binary classification: phishing and safe emails.

DistilBERT is a compact version of BERT that was introduced in 2019 [67]. It reduces BERT's size by 40% while retaining 97% of its performance. This smaller design makes DistilBERT up to 60% faster, offering a more efficient alternative to BERT. The model architecture consists of an encoder, similar to the BERT encoder, that consumes the input text to generate embeddings. A fully connected layer then processes these embeddings to produce predictions. Finally, the output layer uses a softmax activation function for classification, generating probabilities for each class.

Robustly optimized BERT approach (RoBERTa) is an improved variant of BERT [30]. It enhances training using more data, larger mini-batches, and higher learning rates. RoBERTa is pre-trained on five diverse English corpora, including BookCorpus, CC-News, and OpenWebText [68]. These modifications improve its robustness and generalization for other tasks. We have used the base model, which retains BERT's core architecture but adjusts key

hyperparameters for better performance. Recent research utilized RoBERTa for phishing email detection [66,69].

Graph convolutional network (GCN) is a multilayer neural network that acts directly on a graph and generates embedding vectors for nodes based on the features of their neighbors [70]. GCN can achieve end-to-end node and structure feature learning by considering the network as a spectral graph. Furthermore, GCN applies to any graph with arbitrary topology [71]. The GCN architecture employed in our framework has the following components: an input layer that accepts a graph representation of the email data, where the node features are represented as TF-IDF vectors, and the edge index defines the connections between nodes. The architecture includes two GCN layers. The first GCN layer takes input dimensions equal to the size of the TF-IDF vector and outputs 128 hidden features using the ReLU activation function. The second GCN layer processes the 128-dimensional and produces output dimensions corresponding to the number of classes with a log-softmax activation function. Finally, the output layer generates class probabilities for binary classification.

4. Results and Discussion

4.1. Experimental Setup

This work analyzes email body content, identified as the most effective approach [54]. In our proposed framework, we train traditional ML and DL models to compare their effectiveness in classifying phishing attempts. The experiments use Python and `scikit-learn` on Google Colab Pro, leveraging 50 GB of RAM to handle large datasets efficiently. We divided each dataset into training and testing sets using an 80–20 split. All models are trained and tested using the `scikit-learn` library. The parameters used for ML models are as follows: In Naïve Bayes classifier, we set MultinomialNB ($alpha = 0.1$). In logistic regression solvers, Limited-memory Broyden–Fletcher–Goldfarb–Shanno (lbfgs) was applied, and to prevent model overfitting, we used $penalty = L2$. In SGD Classifier, we used $loss = hinge$, which gives a linear SVM, $penalty = l2$, $alpha = 0.0001$, and the $learning_rate =$ "optimal". In XGBoost, $learning_rate$ was set to 0.3, and the Maximum depth of a tree = 6. In Decision Tree Classifier, $criterion =$ "gini" and $splitter = best$. In Random Forest Classifier, the criterion is "gini". In Extra Trees Classifier, the $criterion =$ "mini", $max_features =$ "sqrt". To prevent overfitting, we implemented regularization techniques, `dropout` in DL models and `L2` regularization in ML classifiers. The hyperparameters used for DL models are illustrated in Table 2.

Table 2. Hyperparameters of Deep Learning Models.

Model	Optimizer	Loss Function	Batch Size	Number of Epochs	Learning Rate
CNN	Adam	Binary Crossentropy	32	5	0.01
RNN	Adam	Binary Crossentropy	32	5	0.01
LSTM	Adam	Binary Crossentropy	32	5	0.01
GCN	Adam	Crossentropy Loss	Entire graph is processed in a single batch	10	0.01
BERT	Adam	Binary Crossentropy	8	5	5×10^{-5}
DistilBERT	Adam	Cross Entropy Loss with class weights	8	5	Automatically tuned by DistilBERT model
RoBERTa	Adam	Crossentropy Loss with class weights	8	5	Automatically tuned by RoBERTa model

4.2. Evaluation Metrics

To assess the models' performances on the binary classification task of identifying phishing email texts, we employed several widely recognized metrics, as outlined in [72]:

Accuracy: Calculated as the ratio of correct phishing and safe email predictions to the total number of emails.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}} \tag{1}$$

Precision: The ratio of true identifications of phishing emails that were phishing.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{2}$$

Recall or Sensitivity: The ratio of actual positive phishing emails that were correctly identified as phishing.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{3}$$

F1 Score: A metric that emphasizes the balance between precision and recall.

$$\text{F1-score} = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} \tag{4}$$

or

$$\text{F1-score} = \frac{2 \times \text{TP}}{2 \times \text{TP} + \text{FP} + \text{FN}} \tag{5}$$

The terms TP, TN, FP, and FN denote true positives, true negatives, false positives, and false negatives, respectively. In our study, the positive class is phishing emails, while the negative class is benign emails.

4.3. Results

This study presents significant findings in detecting phishing emails using ML and DL models. Table 3 summarizes the accuracy achieved by all models across the ten datasets. Detailed results, including precision, recall, and F1-Score for each model on each dataset, are available in Appendix A. Below, we provide detailed answers and discussions for all our research questions.

Our experimental results indicate that BERT emerged as the top-performing model, as highlighted in Tables 3 and 4. It achieved the highest accuracy, precision, recall, and F1-Score (see Table 4). This superior performance can be attributed to BERT's bidirectional transformer architecture, which enables a deep understanding of text structure and semantics. Its ability to capture subtle details in text makes it particularly effective in detecting the nuanced tactics used in phishing emails. Furthermore, BERT consistently demonstrated a balance between high precision and high recall, with minimal variance between the two metrics. This balance ensures the model accurately identifies phishing emails and reduces false positives. This addresses our RQ1, identifying the most effective ML and DL models for phishing email detection.

Similarly, RoBERTa, another transformer-based model, delivered outstanding results, particularly on the merged dataset, as shown in Table 3. The merged dataset comprises a substantial volume of data with diverse text structures, derived from multiple sources. The strong performance of BERT and RoBERTa on this dataset underscores the effectiveness of transformer-based approaches for text classification tasks involving complex and heterogeneous data. Their ability to adapt to varied language patterns makes them well-suited for detecting phishing emails across different sources. The accuracies of the balanced dataset are close to those of the unbalanced dataset, likely because the class percentages before balancing (45.81% and 54.19%) were relatively close, resulting in minimal impact from the

balancing technique. This answers our RQ4, why DL models demonstrate superior results in phishing detection.

Table 3. Accuracy Results of All ML and DL Phishing Detection Models on All Datasets.

Model	Enron	SpamAssassin	Ling	TREC_05	TREC_06	TREC_07	CEAS_08	Nazario_5	Nigerian_5	Merged	Merged (Balanced)
Naive Bayes	97.85%	96.70%	98.43%	92.27%	93.16%	96.61%	97.47%	98.82%	99.23%	92.93%	92.93%
Logistic Regression	98.29%	96.25%	97.20%	97.46%	97.30%	99.04%	99.04%	97.97%	98.79%	97.19%	97.00%
SGD Classifier	98.35%	97.68%	99.48%	97.00%	97.74%	99.25%	99.10%	98.82%	99.34%	96.66%	96.51%
XGBoost	97.23%	96.79%	98.43%	95.77%	96.92%	98.80%	98.75%	97.80%	98.68%	95.65%	95.56%
Decision Tree	95.38%	90.98%	94.76%	94.22%	94.82%	97.48%	98.23%	94.42%	95.06%	94.17%	94.13%
Random Forest	98.09%	96.88%	98.60%	96.99%	96.63%	98.96%	99.15%	97.63%	99.34%	97.20%	97.23%
Extra Tree	98.37%	97.23%	97.73%	97.14%	96.95%	98.93%	99.28%	97.97%	98.79%	97.31%	97.38%
CNN	98.34%	98.11%	98.25%	96.57%	97.80%	99.45%	99.38%	98.37%	99.45%	98.07%	98.09%
RNN	96.56%	97.07%	95.45%	94.64%	95.67%	98.04%	99.27%	96.90%	98.18%	95.14%	96.63%
LSTM	98.14%	97.42%	98.43%	97.15%	96.77%	99.27%	99.12%	97.55%	98.82%	97.43%	97.67%
BERT	99.31%	98.88%	100.00%	98.94%	98.54%	99.24%	99.66%	99.51%	99.61%	98.65%	98.99%
DistilBERT	99.16%	98.88%	99.83%	98.85%	98.57%	99.09%	99.68%	99.35%	99.37%	98.74%	98.83%
RoBERTa	99.19%	98.80%	99.83%	98.88%	98.63%	98.91%	98.99%	99.51%	99.45%	99.03%	99.08%
GCN	72.49%	79.78%	86.71%	72.44%	76.17%	72.41%	73.25%	69.98%	72.06%	70.99%	71.11%

Table 4. Best Performing Models on Ten Datasets.

Dataset	Best Model (s)	Accuracy	Precision	Recall	F1-Score
Enron	BERT	99.31%	99.21%	99.31%	99.26%
SpamAssassin	BERT	98.88%	98.23%	97.94%	98.09%
Ling	BERT	100.00%	100.00%	100.00%	100.00%
TREC_05	BERT	98.94%	98.73%	98.71%	98.72%
TREC_06	RoBERTa	98.63%	96.95%	97.08%	97.02%
TREC_07	CNN	99.45%	99.24%	99.76%	99.50%
CEAS_08	DistilBERT	99.68%	99.68%	99.75%	99.71%
Nazario	RoBERTa	99.51%	99.67%	99.33%	99.50%
Nigerian	BERT	99.61%	99.53%	99.69%	99.61%
Merged	RoBERTa	99.03%	99.17%	98.80%	98.98%
Merged (Balanced)	RoBERTa	99.08%	99.20%	98.96%	99.08%

In contrast, Table 3 indicates that the GCN model did not perform well. GCNs are optimized for graph-structured data, while email text is inherently linear. Without transforming the text into a graph representation, GCNs struggle to extract meaningful patterns, which limits their effectiveness in this context.

As part of addressing RQ1 concerning the best-performing ML algorithm, we found out that SGD Classifier emerged as the best-performing traditional machine learning model across the analysis of the ten datasets, having the highest average accuracy of 98.17%. Its consistently high performance, adaptability across various datasets, and balanced precision and recall make it a reliable choice for phishing detection tasks. While other classifiers, such as the Extra Trees, performed well on datasets like Enron, CEAS_08, and the Merged dataset, and Random Forest excelled on the TREC_05 dataset, the overall consistency of the SGD Classifier makes it the preferred option.

Regarding RQ2, which investigates the performance of ML and DL models when applied to multiple individual datasets and a merged dataset, we studied the effectiveness

of each model on each dataset. We compared the results with the merged and balanced merged datasets. Detailed results are provided in Table 3. Moreover, we investigated each model's efficiency on all datasets regarding the training time. As discussed above, DL models demonstrated higher accuracy than traditional ML models. However, their computational demands and longer training times present challenges, as shown in Table 5. DL models typically require GPUs for efficient processing, making them less practical for real-time or large-scale applications compared to traditional ML models. To investigate this further, we measured the prediction time of the top-performing DL models (i.e., BERT, RoBERTa, and DistilBERT) on the merged and balanced merged datasets, which encompass all data samples from the 9 datasets. This will help in understanding which top-performing model is the best candidate for real-time phishing detection applications. As shown in Table 6, DistilBERT has the fastest prediction time across the two datasets, which makes it suitable for real-time phishing detection applications.

Table 5. Training Time Results of Phishing Detection Models (in Seconds).

Model	Enron	SpamAssassin	Ling	TREC_05	TREC_06	TREC_07	CEAS_08	Nazario_5	Nigerian_5	Merged	Balanced Merged
Naive Bayes	0.4045	0.0774	0.4096	0.7002	0.2044	0.6891	0.5992	0.0414	0.0639	2.7606	2.4230
Logistic Regression	6.0422	4.3615	0.4096	14.0869	5.8902	4.6597	4.5619	2.2404	2.6391	14.9105	20.9376
SGD Classifier	4.7780	0.8779	0.4595	9.3601	3.0694	6.7983	5.3173	0.5486	0.6771	25.6718	23.2197
XGBoost	32.8451	16.4625	24.9872	42.6082	19.8026	38.7431	29.1080	7.3258	22.6479	86.8558	88.0840
Decision Tree	186.7102	22.4780	4.4804	684.2861	30.6576	407.0792	250.7007	3.9194	7.9171	8523.5984	7494.3420
Random Forest	92.4306	11.3588	4.0028	293.0442	33.9499	249.9548	170.5102	3.2221	7.2300	1994.3688	2057.4461
Extra Tree	251.8828	33.3333	11.6939	683.8933	108.0892	527.6884	421.4103	7.7123	19.4767	4287.0341	4209.5902
CNN	1394.9935	250.6515	53.4167	2873.5432	332.5567	3395.9233	999.6245	73.8962	60.3413	33,641.1476	37,260.0418
RNN	1198.6864	200.5724	97.6607	2051.2652	412.7603	3983.1043	1148.2347	105.0173	62.5940	39,072.5457	32,016.6140
LSTM	1616.3018	415.1188	109.0630	2594.2131	1802.2137	4897.9552	1454.6466	201.7589	363.3502	35,495.7471	25,345.1311
BERT	705.5716	283.1131	156.4819	1290.6487	388.8494	1106.4427	877.7004	96.2193	150.1970	5137.9657	4626.4871
DistilBERT	420.9068	164.1796	85.1270	721.0618	218.7989	615.7078	493.6949	60.7465	84.3074	2839.2592	2751.6064
RoBERTa	713.6597	298.7486	164.4693	1336.9023	396.7849	1168.1359	937.3521	82.6740	152.0265	5062.8701	5013.7805
GCN	1456.7544	315.2311	101.6573	2437.8352	566.2911	4258.3391	1345.6267	122.2786	200. 3135	34,604.4459	24,417.8203

Table 6. Prediction Time (in Seconds) of Transformer-based Models for Merged and Balanced Merged Datasets.

Model	Merged	Balanced Merged
BERT	0.0364	0.0329
DistilBERT	0.0291	0.0258
RoBERTa	0.0379	0.0340

The Ling dataset achieves the highest average accuracy among all datasets but exhibits lower recall values. This indicates that while models perform well in overall predictions on this dataset, they tend to miss a more significant number of phishing emails—a critical shortcoming in security applications. In contrast, the TREC_07 dataset achieves the highest recall, showing that models are most effective at identifying phishing emails in this dataset. Furthermore, the TREC_07 dataset also excels in precision, ensuring that phishing emails are accurately classified with minimal false positives. The TREC_05 dataset, however, demonstrates the best balance between precision and recall, with the slightest difference between these metrics. Appendices A and B provide detailed experimental results for all datasets and models.

Regarding RQ3, both ML and DL models have their respective strengths and weaknesses. Although DL models like BERT and RoBERTa offer superior accuracy, their high

resource requirements make them less feasible for large-scale applications. However, DistilBERT—a more lightweight version of BERT—also delivers strong results, suggesting potential for efficiency improvements. Future work could focus on optimizing these models for practical, real-world deployments. False positives (safe emails incorrectly flagged as phishing, FPs) and false negatives (phishing emails mistakenly classified as safe, FNs) can significantly impact the effectiveness of real-world email security systems. Minimizing these errors is essential to ensure both security and user experience. For best-performing models across all datasets, we have provided the number of FPs (to evaluate the impact on user experience) and FNs (to assess the potential security threats to users). Detailed results are shown in Appendix B. Because the studied datasets span different periods and phishing techniques evolve, older emails may not accurately reflect current phishing strategies. Future work can include a simulation of sophisticated phishing attacks to estimate the models' effectiveness in detecting evasive threats while minimizing FPs.

We did not use a dedicated validation set in our experimental setup, and as a result, we do not have a training-validation accuracy and loss plot. Although we did not explicitly use a validation set test, our approach still provides strong evidence of generalization in some models. First, the high test accuracy, especially on the merged dataset, suggests strong generalization since the merged dataset comes from different sources. Models that achieve high accuracy on this dataset tend to generalize well across diverse phishing scenarios. Second, we have tested the top-performing ML and DL models—namely, SGD Classifier, BERT, DistilBERT, and RoBERTa—on an external dataset (`phishing validation emails` dataset [73]) to validate the models' robustness further. This dataset includes real and synthetic phishing email samples. The results of this experiment are summarized in Table 7. In this experiment, we have trained these models on each of the 11 datasets separately and then evaluated their performance on the external dataset. BERT achieved a test accuracy of 81.30% and 88.80% on the external dataset, which is lower than its performance when tested on the merged-balanced and merged datasets. This discrepancy may be attributed to BERT's Next Sentence Prediction (NSP) mechanism, which is less effective for certain tasks and can limit generalizability. In contrast, RoBERTa, which benefits from extended training and larger datasets, demonstrated superior performance. Therefore, the fact that RoBERTa maintains high accuracy on unseen data supports its ability to generalize beyond the training set. Third, the small accuracy gap between the test set from the merged-balanced dataset (99.08%) and the external test set (95.00%) in RoBERTa indicates that the model is not overfitting.

Table 7. Testing Accuracy Results of SGD Classifier and Transformer Models on an External Dataset (Phishing Validation Emails Dataset [73]) When Trained Separately on Each Dataset.

Model	Enron	SpamAssassin	Ling	TREC_05	TREC_06	TREC_07	CEAS_08	Nazario_5	Nigerian_5	Merged	Merged (Balanced)
SGD Classifier	69.60%	55.85%	64.90%	73.50%	77.75%	50.80%	69.55%	85.80%	50.00%	77.35%	77.15%
BERT	82.45%	55.30%	71.90%	69.65%	90.65%	62.50%	75.90%	79.70%	61.60%	88.80%	81.30%
DistilBERT	75.60%	60.90%	71.05%	70.90%	91.95%	51.90%	73.60%	75.70%	61.80%	85.75%	80.40%
RoBERTa	88.80%	50.85%	74.85%	70.25%	92.10%	60.05%	75.15%	80.35%	65.00%	**96.00%**	**95.00%**

4.4. Practical Strategies for Implementing AI-Driven Techniques in Phishing Email Detection

From the experiments we conducted, we provide actionable insights for researchers and developers who are adopting ML and DL models for phishing email detection. First, transformer-based models such as BERT and RoBERTa should be prioritized, as they consistently achieve high accuracy (e.g., 98–99% on the merged dataset). This is due to their advanced architecture, which understands context and semantics in text, making them highly effective for analyzing email content and recognizing phishing attempts. When

trained on large datasets, these models are exceptionally robust, as they can generalize well to diverse phishing tactics. Second, dataset preprocessing steps, such as balancing datasets, are essential to ensure optimal model performance. Specifically, datasets should be balanced and represent real-world email traffic to improve model performance and reduce bias. Third, updating models regularly with new phishing samples is crucial for the adaptability to evolving attack techniques and maintaining high detection accuracy. Fourth, integrating these models into real-world email security systems requires careful consideration of computational resources and deployment strategies, as transformer-based models can be resource-intensive. Developers should ensure that the AI detection system is scalable to handle large volumes of emails without compromising speed or accuracy. Secure Email Gateway (SEG) is a widely used email security defense solution to filter out malicious email traffic such as phishing and spam emails. Generally, SEGs detect emails with harmful content such as spam, phishing links, or malware using signature analysis, URL scanning, machine learning, custom policies, etc. Once identified, SEGs block malicious emails from reaching the recipient. Integrating deep learning (DL) models like BERT and RoBERTa into existing email security gateways (e.g., Microsoft Defender [74], Gmail security filters) is feasible. However, key factors such as detection accuracy, adaptability, latency, and robustness against adversarial attacks must be considered for successful integration. Our experimental results indicate that RoBERTa and BERT are strong candidates due to their high detection accuracy and extensive pretraining. Notably, RoBERTa's robust pretraining allows it to generalize better and requires less data for fine-tuning, making it an ideal choice for SEG integration. However, lightweight variants such as DistilBERT may be preferable for real-time detection due to their lower latency.

By leveraging these insights, researchers and developers can build more accurate and reliable phishing detection systems, ultimately enhancing email security and reducing the risk of cyberattacks.

5. Conclusions and Future Work

Phishing email attacks are a prevalent cybersecurity threat designed to steal sensitive information, including passwords and login credentials. Early phishing detection methods were relatively simple, but the increasing sophistication of phishing techniques has necessitated the use of advanced AI technologies for accurate detection. This study presents a comprehensive framework for evaluating phishing email detection models using diverse datasets and models. The results demonstrate that transformer-based models offer superior performance over traditional models. Specifically, BERT and RoBERTa delivered the best results on the merged dataset. Furthermore, DistilBERT demonstrated competitive accuracy with reduced computational overhead, making it suitable for real-time detection systems. Future work could focus on optimizing these models for practical, real-world deployments. Furthermore, expanding the research to include phishing email datasets in languages other than English would provide valuable insights into the cross-linguistic performance of ML and DL models, further enhancing their global applicability and effectiveness. Additionally, future research could focus on assessing adversarial robustness and how well AI-driven phishing detection models withstand evasion techniques employed by sophisticated synthetic and real phishing attacks.

Author Contributions: Conceptualization, A.A. (Abeer Alhuzali), M.A. and A.A. (Ahad Alloqmani); methodology, A.A. (Abeer Alhuzali), M.A. and A.A. (Ahad Alloqmani); software, M.A. and A.A. (Ahad Alloqmani); validation, A.A. (Abeer Alhuzali), M.A. and A.A. (Ahad Alloqmani); formal analysis, A.A. (Abeer Alhuzali), M.A. and A.A. (Ahad Alloqmani); investigation, A.A. (Abeer Alhuzali), M.A. and A.A. (Ahad Alloqmani); resources, A.A. (Abeer Alhuzali), M.A. and A.A. (Ahad Alloqmani); data curation, M.A. and A.A. (Ahad Alloqmani); writing—original draft preparation,

A.A. (Abeer Alhuzali), M.A., A.A. (Ahad Alloqmani) and F.A.; writing—review and editing, A.A. (Abeer Alhuzali), M.A., A.A. (Ahad Alloqmani) and F.A.; visualization, A.A. (Abeer Alhuzali), M.A., A.A. (Ahad Alloqmani) and F.A.; supervision, A.A. (Abeer Alhuzali) and F.A.; project administration, A.A. (Abeer Alhuzali); funding acquisition, A.A. (Abeer Alhuzali). All authors have read and agreed to the published version of the manuscript.

Funding: This research work was funded by Institutional Fund Projects under grant no. (IFPIP: 571-612-1443). The authors gratefully acknowledge the technical and financial support from the Ministry of Education and King Abdulaziz University, DSR, Jeddah, Saudi Arabia.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The raw data supporting the conclusions of this article will be made available by the authors on request.

Acknowledgments: We would like to thank the reviewers for their valuable comments, which have contributed to improving the quality of this work.

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A. Experiments Results

The following tables, from Table A1 to Table A11, present detailed performance metrics for each model on each dataset.

Table A1. Performance Metrics for Phishing Detection Models On Enron Dataset (Training Time in Seconds denoted as **S**).

Model	Accuracy	Precision	Recall	F1-Score	Training Time (S)
Naive Bayes	97.85%	97.86%	97.58%	97.72%	0.4045 S
Logistic Regression	98.29%	97.11%	99.32%	98.21%	6.0422 S
SGD Classifier	98.35%	97.11%	99.11%	98.27%	4.7780 S
XGBoost	97.23%	95.46%	98.83%	97.12%	32.8451 S
Decision Tree	95.38%	94.93%	95.30%	95.12%	186.7102 S
Random Forest	98.09%	97.37%	98.61%	97.99%	92.4306 S
Extra Tree	98.37%	97.98%	98.58%	98.28%	251.8828 S
CNN	98.34%	98.21%	98.25%	98.23%	1394.9935 S
RNN	96.56%	98.11%	94.49%	96.26%	1198.6864 S
LSTM	98.14%	98.27%	97.75%	98.01%	1616.3018 S
BERT	99.31%	99.21%	99.31%	99.26%	705.5716 S
DistilBERT	99.16%	99.03%	99.17%	99.10%	420.9068 S
RoBERTa	99.19%	99.24%	99.02%	99.13%	713.6597 S
GCN	72.49%	72.45%	72.49%	72.43%	1456.7544 S

Table A2. Performance Metrics for Phishing Detection Models On SpamAssassin Dataset (Training Time in Seconds denoted as **S**).

Model	Accuracy	Precision	Recall	F1-Score	Training Time (S)
Naive Bayes	96.70%	95.58%	93.64%	94.60%	0.0774 S
Logistic Regression	96.25%	97.80%	89.88%	93.67%	4.3615 S
SGD Classifier	97.68%	97.80%	96.24%	96.24%	0.8779 S
XGBoost	96.79%	96.13%	93.35%	94.72%	16.4625 S
Decision Tree	90.98%	83.56%	88.15%	85.79%	22.4780 S
Random Forest	96.88%	95.07%	94.80%	94.93%	11.3588 S
Extra Tree	97.23%	96.19%	94.80%	95.49%	33.3333 S
CNN	98.11%	95.74%	97.97%	96.84%	250.6515 S
RNN	97.07%	94.80%	95.35%	95.07%	200.5724 S
LSTM	97.42%	95.38%	95.93%	95.65%	415.1188 S
BERT	98.88%	98.23%	97.94%	98.09%	283.1131 S
DistilBERT	98.88%	99.40%	96.76%	98.06%	164.1796 S
RoBERTa	98.80%	97.94%	97.94%	97.94%	298.7486 S
GCN	79.78%	80.13%	79.78%	77.24%	315.2311 S

Table A3. Performance Metrics for Phishing Detection Models On Ling Dataset (Training Time in Seconds denoted as **S**).

Model	Accuracy	Precision	Recall	F1-Score	Training Time (S)
Naive Bayes	98.43%	100.00%	90.22%	94.86%	0.4096 S
Logistic Regression	97.20%	100.00%	82.61%	90.48%	0.4096 S
SGD Classifier	99.48%	100.00%	97.83%	98.36%	0.4595 S
XGBoost	98.43%	98.82%	91.30%	94.92%	24.9872 S
Decision Tree	94.76%	81.00%	88.04%	84.38%	4.4804 S
Random Forest	98.60%	98.84%	92.39%	95.51%	4.0028 S
Extra Tree	97.73%	100.00%	85.87%	92.40%	11.6939 S
CNN	98.25%	92.71%	96.74%	94.68%	53.4167 S
RNN	95.45%	83.00%	90.22%	86.46%	97.6607 S
LSTM	98.43%	93.68%	96.74%	95.19%	109.0630 S
BERT	100.00%	100.00%	100.00%	100.00%	156.4819 S
DistilBERT	99.83%	100.00%	98.68%	99.34%	85.1270 S
RoBERTa	99.83%	98.70%	100.00%	99.35%	164.4693 S
GCN	86.71%	75.19%	86.71%	80.54%	101.6573 S

Table A4. Performance Metrics for Phishing Detection Models On TREC_05 Dataset (Training Time in Seconds denoted as **S**).

Model	Accuracy	Precision	Recall	F1-Score	Training Time (S)
Naive Bayes	92.27%	95.14%	84.53%	89.52%	0.7002 S
Logistic Regression	97.46%	95.40%	98.22%	96.79%	14.0869 S
SGD Classifier	97.00%	95.40%	98.24%	96.24%	9.3601 S
XGBoost	95.77%	92.21%	97.39%	94.73%	42.6082 S
Decision Tree	94.22%	92.52%	92.68%	92.60%	684.2861 S
Random Forest	96.99%	96.65%	95.61%	96.12%	293.0442 S
Extra Tree	97.14%	97.31%	95.31%	96.30%	683.8933 S
CNN	96.57%	97.68%	96.89%	97.84%	2873.5432 S
RNN	94.64%	96.59%	90.34%	93.36%	2051.2652 S
LSTM	97.15%	97.84%	95.27%	96.54%	2594.2131 S
BERT	98.94%	98.73%	98.71%	98.72%	1290.6487 S
DistilBERT	98.85%	98.75%	98.47%	98.61%	721.0618 S
RoBERTa	98.88%	99.01%	98.28%	98.64%	1336.9023 S
GCN	72.44%	72.26%	72.44%	72.31%	2437.8352 S

Table A5. Performance Metrics for Phishing Detection Models On TREC_06 Dataset (Training Time in Seconds denoted as **S**).

Model	Accuracy	Precision	Recall	F1-Score	Training Time (S)
Naive Bayes	93.16%	96.27%	72.57%	82.76%	0.2044 S
Logistic Regression	97.30%	96.58%	91.28%	93.85%	5.8902 S
SGD Classifier	97.74%	96.58%	95.22%	95.02%	3.0694 S
XGBoost	96.92%	93.00%	93.39%	93.19%	19.8026 S
Decision Tree	94.82%	87.74%	89.59%	88.66%	30.6576 S
Random Forest	96.63%	96.04%	88.75%	92.25%	33.9499 S
Extra Tree	96.95%	97.09%	89.17%	92.96%	108.0892 S
CNN	97.80%	96.83%	95.80%	97.81%	332.5567 S
RNN	95.67%	91.43%	90.51%	90.97%	412.7603 S
LSTM	96.77%	96.80%	96.77%	96.78%	1802.2137 S
BERT	98.54%	96.82%	96.82%	96.82%	388.8494 S
DistilBERT	98.57%	96.57%	97.21%	96.89%	218.7989 S
RoBERTa	98.63%	96.95%	97.08%	97.02%	396.7849 S
GCN	76.17%	78.18%	76.17%	66.25%	566.2911 S

Table A6. Performance Metrics for Phishing Detection Models On TREC_07 Dataset (Training Time in Seconds denoted as **S**).

Model	Accuracy	Precision	Recall	F1-Score	Training Time (S)
Naive Bayes	96.61%	98.89%	94.76%	96.78%	0.6891 S
Logistic Regression	99.04%	98.71%	99.51%	99.11%	4.6597 S
SGD Classifier	99.25%	98.71%	99.60%	99.31%	6.7983 S
XGBoost	98.80%	98.48%	99.30%	98.89%	38.7431 S
Decision Tree	97.48%	97.60%	97.72%	97.66%	407.0792 S
Random Forest	98.96%	99.01%	99.06%	99.03%	249.9548 S
Extra Tree	98.93%	99.32%	98.69%	99.01%	527.6884 S
CNN	99.45%	99.24%	99.76%	99.50%	3395.9233 S
RNN	98.04%	98.78%	97.62%	98.20%	3983.1043 S
LSTM	99.27%	99.42%	99.25%	99.34%	4897.9552 S
BERT	99.24%	99.38%	99.23%	99.30%	1106.4427 S
DistilBERT	99.09%	99.11%	99.23%	99.17%	615.7078 S
RoBERTa	98.91%	99.09%	98.92%	99.00%	1168.1359 S
GCN	72.41%	72.35%	72.41%	72.37%	4258.3391 S

Table A7. Performance Metrics for Phishing Detection Models On CEAS_08 Dataset (Training Time in Seconds denoted as **S**).

Model	Accuracy	Precision	Recall	F1-Score	Training Time (S)
Naive Bayes	97.47%	99.57%	95.92%	97.71%	0.5992 S
Logistic Regression	99.04%	98.66%	99.66%	99.16%	4.5619 S
SGD Classifier	99.10%	98.66%	99.66%	99.20%	5.3173 S
XGBoost	98.75%	98.28%	99.52%	98.90%	29.1080 S
Decision Tree	98.23%	97.91%	98.97%	98.44%	250.7007 S
Random Forest	99.15%	99.00%	99.50%	99.25%	170.5102 S
Extra Tree	99.28%	99.25%	99.47%	99.36%	421.4103 S
CNN	99.39%	99.59%	99.31%	99.45%	999.6245 S
RNN	99.27%	99.18%	99.52%	99.35%	1148.2347 S
LSTM	99.12%	98.75%	99.68%	99.21%	1454.6466 S
BERT	99.66%	99.72%	99.65%	99.69%	877.7004 S
DistilBERT	99.68%	99.68%	99.75%	99.71%	493.6949 S
RoBERTa	98.99%	99.21%	98.96%	99.09%	937.3521 S
GCN	73.25%	73.15%	73.25%	73.13%	1345.6267 S

Table A8. Performance Metrics for Phishing Detection Models On Nazario_5 Dataset (Training Time in Seconds denoted as **S**).

Model	Accuracy	Precision	Recall	F1-Score	Training Time (S)
Naive Bayes	98.82%	97.53%	100.00%	98.75%	0.0414 S
Logistic Regression	97.97%	99.25%	96.38%	97.79%	2.2404 S
SGD Classifier	98.82%	99.25%	97.83%	98.72%	0.5486 S
XGBoost	97.80%	98.88%	96.38%	97.61%	7.3258 S
Decision Tree	94.42%	92.63%	95.65%	94.12%	3.9194 S
Random Forest	97.63%	95.17%	100.00%	97.53%	3.2221 S
Extra Tree	97.97%	95.83%	100.00%	97.87%	7.7123 S
CNN	98.37%	98.10%	98.72%	98.41%	73.8962 S
RNN	96.90%	96.82%	97.12%	96.97%	105.0173 S
LSTM	97.55%	96.27%	99.04%	97.64%	201.7589
BERT	99.51%	99.34%	99.67%	99.50%	96.2193 S
DistilBERT	99.35%	99.01%	99.67%	99.34%	60.7465 S
RoBERTa	99.51%	99.67%	99.33%	99.50%	82.6740 S
GCN	69.98%	70.06%	69.98%	69.91%	122.2786 S

Table A9. Performance Metrics for Phishing Detection Models On Nigerian_5 Dataset (Training Time in Seconds denoted as **S**).

Model	Accuracy	Precision	Recall	F1-Score	Training Time (S)
Naive Bayes	99.23%	100.00%	97.73%	98.85%	0.0639 S
Logistic Regression	98.79%	100.00%	96.43%	98.18%	2.6391 S
SGD Classifier	99.34%	100.00%	98.05%	99.02%	0.6771 S
XGBoost	98.68%	99.33%	96.75%	98.03%	22.6479 S
Decision Tree	95.06%	89.25%	97.08%	93.00%	7.9171 S
Random Forest	99.34%	99.67%	98.38%	99.02%	7.2300 S
Extra Tree	98.79%	97.75%	98.70%	98.22%	19.4767 S
CNN	99.45%	99.55%	99.40%	99.47%	60.3413 S
RNN	98.18%	97.35%	99.25%	98.29%	62.5940 S
LSTM	98.82%	99.39%	98.35%	98.87%	363.3502 S
BERT	99.61%	99.53%	99.69%	99.61%	150.1970 S
DistilBERT	99.37%	99.37%	99.37%	99.37%	84.3074 S
RoBERTa	99.45%	99.69%	99.22%	99.45%	152.0265 S
GCN	72.06%	72.16%	72.06%	72.04%	200. 3135 S

Table A10. Performance Metrics for Phishing Detection Models On Merged Datasets (Training Time in Seconds denoted as **S**).

Model	Accuracy	Precision	Recall	F1-Score	Training Time (S)
Naive Bayes	92.93%	95.84%	88.49%	92.02%	2.7606 S
Logistic Regression	97.19%	96.77%	97.15%	96.96%	14.9105 S
SGD Classifier	96.66%	96.77%	97.01%	96.40%	25.6718 S
XGBoost	95.65%	93.57%	97.25%	95.37%	86.8558 S
Decision Tree	94.17%	93.72%	93.63%	93.67%	8523.5984 S
Random Forest	97.20%	97.95%	95.94%	96.93%	1994.3688 S
Extra Tree	97.31%	98.58%	95.55%	97.04%	4287.0341 S
CNN	98.07%	97.33%	98.65%	97.99%	33,641.1476 S
RNN	95.14%	96.20%	93.50%	94.83%	39,072.5457 S
LSTM	97.43%	98.04%	96.53%	97.28%	35,495.7471 S
BERT	98.65%	98.61%	98.57%	98.59%	5137.9657 S
DistilBERT	98.74%	98.89%	98.46%	98.67%	2839.2592 S
RoBERTa	99.03%	99.17%	98.80%	98.98%	5062.8701 S
GCN	70.99%	70.88%	70.99%	70.88%	34,604.4459 S

Table A11. Performance Metrics for Phishing Detection Models On Balanced Merged Dataset (Training Time in Seconds denoted as **S**).

Model	Accuracy	Precision	Recall	F1-Score	Training Time (S)
Naive Bayes	92.93%	95.92%	89.74%	92.73%	2.4230 S
Logistic Regression	97.00%	96.50%	97.56%	97.03%	20.9376 S
SGD Classifier	96.51%	96.50%	97.45%	96.56%	23.2197 S
XGBoost	95.56%	93.74%	97.68%	95.67%	88.0840 S
Decision Tree	94.13%	93.86%	94.50%	94.18%	7494.3420 S
Random Forest	97.23%	98.06%	96.39%	97.22%	2057.4461 S
Extra Tree	97.38%	98.55%	96.21%	97.36%	4209.5902 S
CNN	98.09%	97.91%	98.33%	98.12%	37,260.0418 S
RNN	96.36%	96.05%	96.83%	96.44%	32,016.6140 S
LSTM	97.67%	97.65%	97.76%	97.71%	25,345.1311 S
BERT	98.99%	99.29%	98.69%	98.99%	4626.4871 S
DistilBERT	98.83%	99.03%	98.64%	98.83%	2751.6064 S
RoBERTa	99.08%	99.20%	98.96%	99.08%	5013.7805 S
GCN	71.11%	72.21%	72.59%	72.87%	24,417.8203 S

Appendix B. Graphical Representation of Experiments Results

Appendix B.1. Machine Learning Models

This section will present the confusion matrix corresponding to the highest accuracy achieved by the ML models for each dataset. The confusion matrix provides valuable insights into the model's performances by highlighting where errors occur, specifically identifying false positives (FPs) and negatives (FNs). For instance, Figure A1 reveals that Extra Tree classifier incorrectly classified 57 phishing emails as `safe_emails` (missed phish-

ing emails, FNs) and misclassified 40 safe emails as `phishing_emails` (false alarms, FPs). The Extra Tree classifier achieves the highest accuracy and F1-Score among the traditional classifiers on the Enron dataset. It also maintains a strong balance between precision and recall, as shown in Figure A1. The SGD Classifier outperforms other traditional models with the highest accuracy and F1-Score in the SpamAssassin dataset. Its precision and recall are well-balanced, as shown in Figure A2. The SGD Classifier achieves the highest accuracy and a perfect precision score, indicating it correctly identified all emails that were marked as phishing in the Ling dataset, as shown in Figure A3. The logistic regression classifier provides the highest accuracy and F1-Score among the traditional classifiers, reflecting a strong balance between precision and recall in TREC_05 dataset, as presented in Figure A4. As shown in Figure A5, the SGD Classifier attains the highest accuracy and F1-Score among the ML seven models in TREC_06 dataset. The SGD Classifier attains the highest accuracy and F1-Score among the ML seven models in TREC_07 dataset, as shown in Figure A6. The Extra Tree classifier excels with the highest accuracy and F1-Score among traditional models on the CEAS_08 dataset. The confusion matrix is shown in Figure A7. In Nazario_5 dataset, naive Bayes achieves the highest accuracy and a perfect recall, meaning it identified all phishing emails correctly, as shown in Figure A8. In Nigerian_5 dataset, the SGD Classifier attains the highest accuracy and F1-Score, making it the top performer among traditional classifiers, as presented in Figure A9. Among the traditional machine learning classifiers, the Extra Tree Classifier demonstrates the best performance on the merged dataset as illustrated in Figure A10. Among the traditional machine learning classifiers, the Extra Tree Classifier shows the best performance on the balanced merged dataset, as presented in Figure A11.

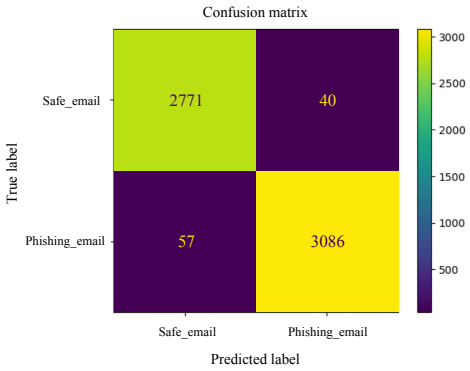


Figure A1. Confusion Matrix of Extra Tree Classifier in Enron Dataset.

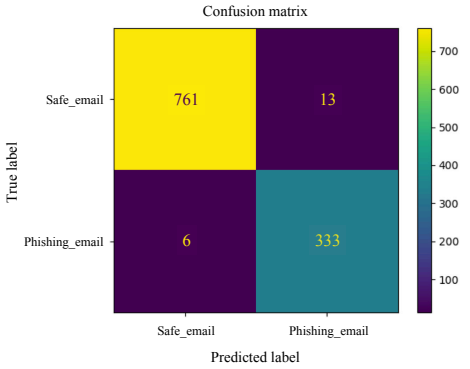


Figure A2. Confusion Matrix of SGD Classifier in SpamAssassin Dataset.

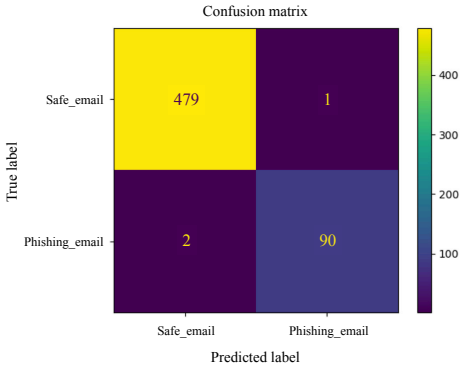


Figure A3. Confusion Matrix of SGD Classifier in Ling Dataset.

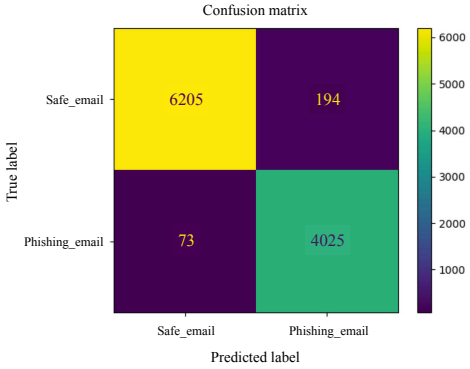


Figure A4. Confusion Matrix of Logistic Regression Classifier in TREC_05 Dataset.

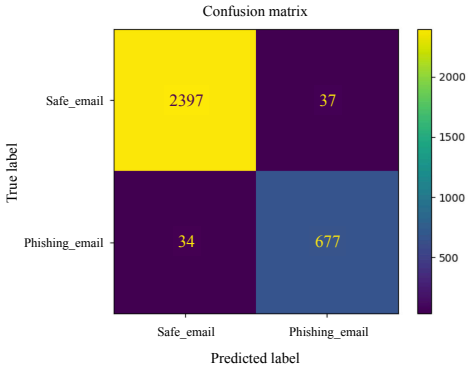


Figure A5. Confusion Matrix of SGD Classifier in TREC_06 Dataset.

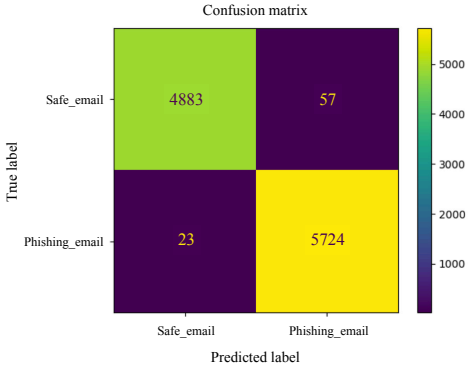


Figure A6. Confusion Matrix of SGD Classifier in TREC_07 Dataset.

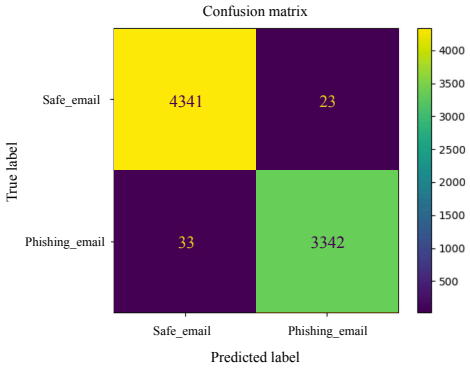


Figure A7. Confusion Matrix of Extra Tree Classifier in CEAS_08 Dataset.

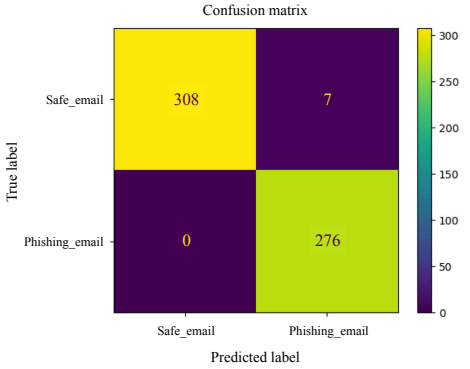


Figure A8. Confusion Matrix of Naive Bayes Classifier in Nazario_5 Dataset.

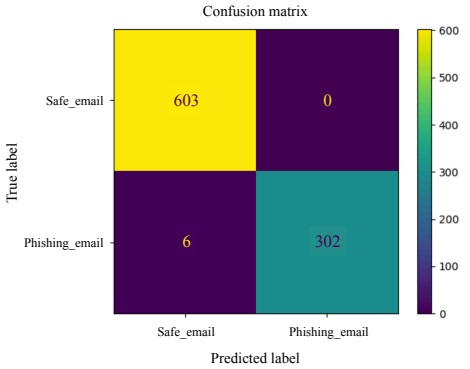


Figure A9. Confusion Matrix of SGD Classifier in Nigerian_5 Dataset.

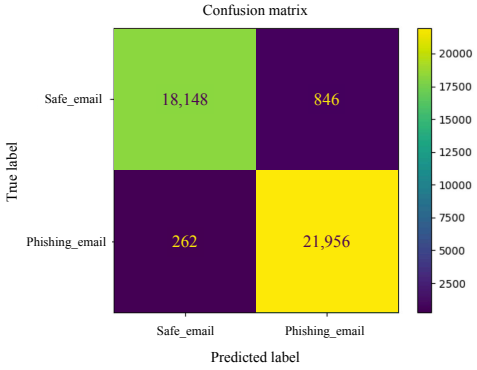


Figure A10. Confusion Matrix of Extra Tree Classifier in the Merged Dataset.

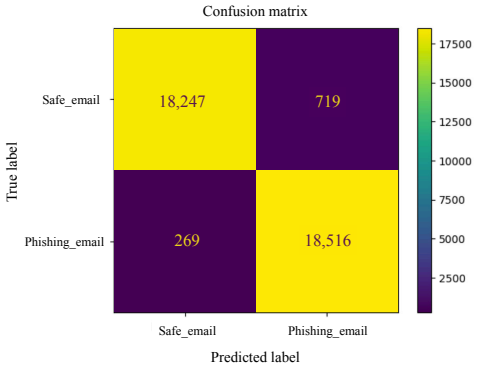


Figure A11. Confusion Matrix of Extra Tree Classifier in Balanced Merged Dataset.

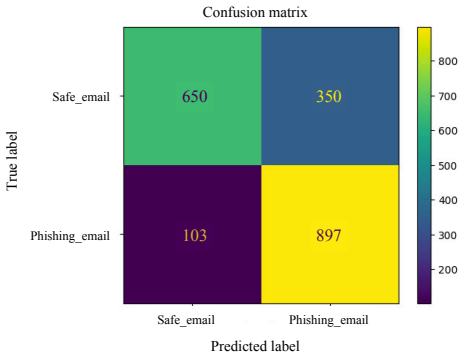


Figure A12. Confusion Matrix of SGD Classifier When Trained on the Merged Dataset and Tested on [73].

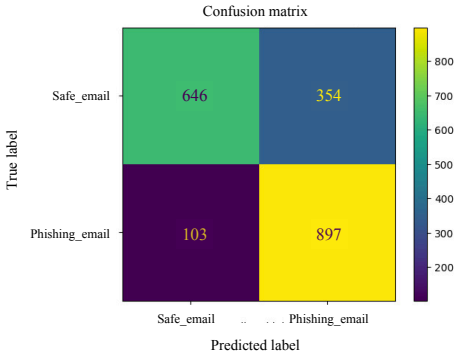


Figure A13. Confusion Matrix of SGD Classifier When Trained on the Balanced Merged Dataset and Tested on [73].

Appendix B.2. Deep Learning Models

This section presents the confusion matrix corresponding to the highest accuracy achieved by DL models on each dataset, as depicted in Figures A14–A24. Figures A25 and A26 show the confusion matrix of RoBERTa when trained on the Merged and Merged Balanced datasets and tested on [73].

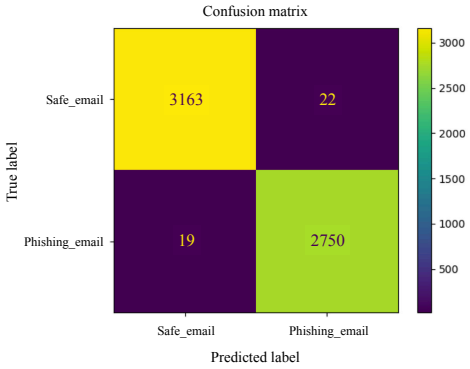


Figure A14. Confusion Matrix of BERT model in Enron Dataset.

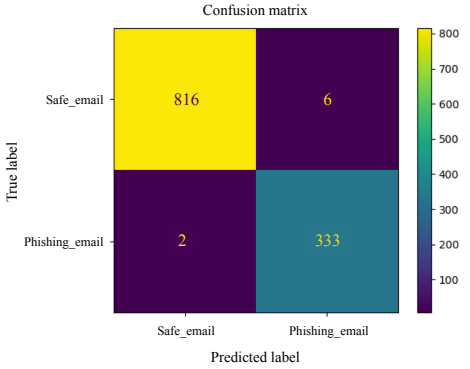


Figure A15. Confusion Matrix of BERT model in SpamAssassin Dataset.

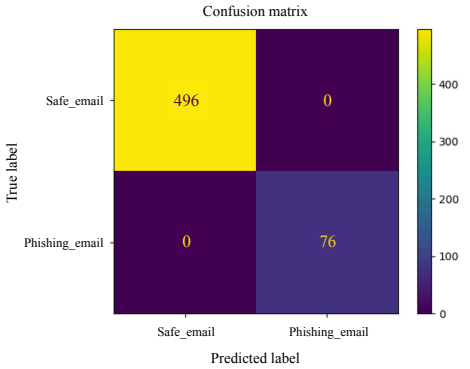


Figure A16. Confusion Matrix of BERT model in Ling Dataset.

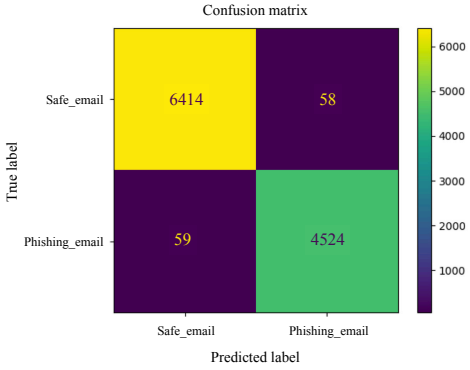


Figure A17. Confusion Matrix of BERT model in TREC_05 Dataset.

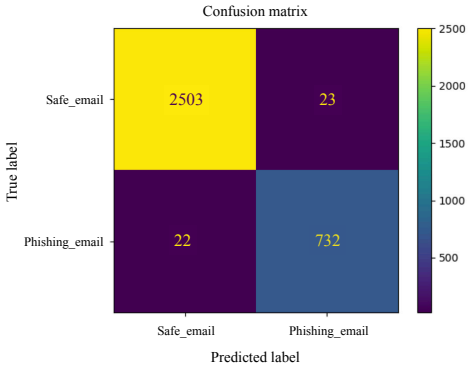


Figure A18. Confusion Matrix of RoBERTa model in TREC_06 Dataset.

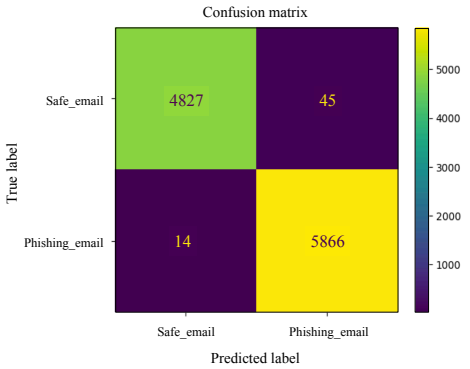


Figure A19. Confusion Matrix of CNN model in TREC_07 Dataset.

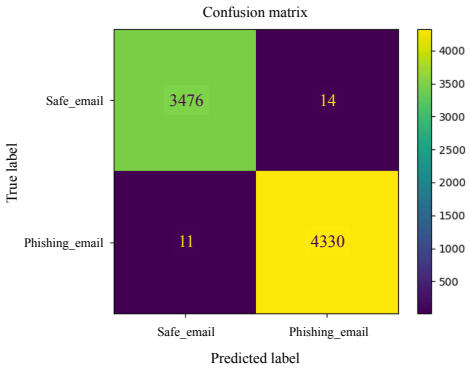


Figure A20. Confusion Matrix of DistilBERT model in CEAS_08 Dataset.

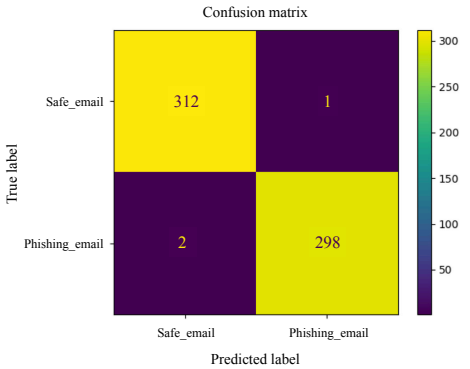


Figure A21. Confusion Matrix of RoBERTa model in Nazario_5 Dataset.

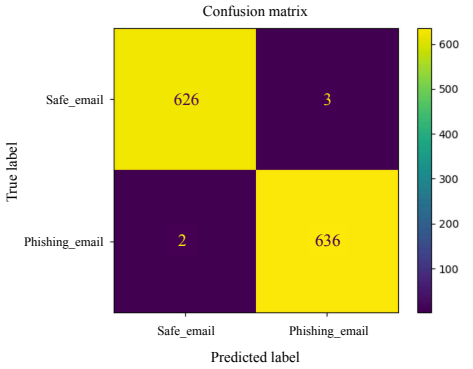


Figure A22. Confusion Matrix of BERT model in Nigerian_5 Dataset.

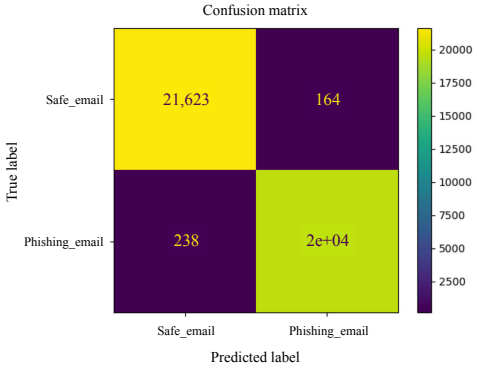


Figure A23. Confusion Matrix of RoBERTa model in the Merged Dataset.

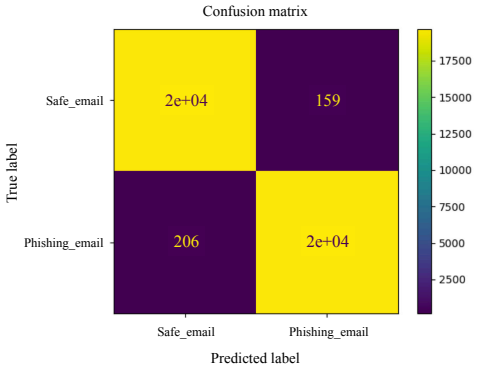


Figure A24. Confusion Matrix of RoBERTa in the Balanced Merged Dataset.

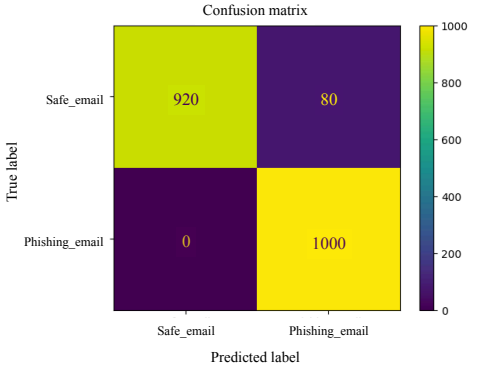


Figure A25. Confusion Matrix of RoBERTa When Trained on the Merged Dataset and Tested on [73].

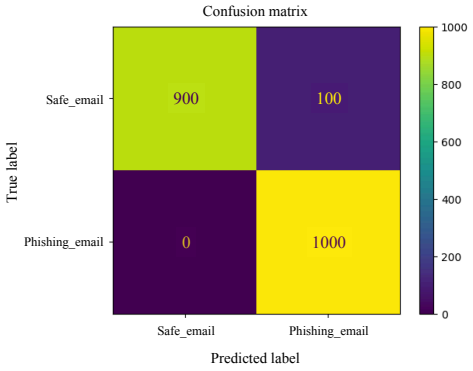


Figure A26. Confusion Matrix of RoBERTa When Trained on the Balanced Merged Dataset and Tested on [73].

References

1. Federal Bureau of Investigation (FBI). Business Email Compromise. 2023. Available online: https://www.fbi.gov/how-we-can-help-you/scams-and-safety/common-frauds-and-scams/business-email-compromise (accessed on 6 March 2025).
2. ZDNet. Hackers Are Targeting Billions of Gmail Users with a Realistic AI Scam: How You Can Stay Safe. 2023. Available online: https://www.zdnet.com/article/hackers-are-targeting-billions-of-gmail-users-with-a-realistic-ai-scam-how-you-can-stay-safe/ (accessed on 6 March 2025).
3. AAG IT. The Latest Phishing Statistics. 2023. Available online: https://aag-it.com/the-latest-phishing-statistics/ (accessed on 6 March 2025).
4. Verizon. Data Breach Investigations Report. 2023. Available online: https://www.verizon.com/business/resources/reports/dbir/ (accessed on 6 March 2025).
5. VIPRE. Email Threats: Latest Trends Q2 2024. 2024. Available online: https://vipre.com/resources/email-threats-latest-trends-q2-2024 (accessed on 6 March 2025).
6. Bergholz, A.; De Beer, J.; Glahn, S.; Moens, M.F.; Paaß, G.; Strobel, S. New filtering approaches for phishing email. *J. Comput. Secur.* **2010**, *18*, 7–35.
7. Fette, I.; Sadeh, N.; Tomasic, A. Learning to detect phishing emails. In Proceedings of the 16th International Conference on World Wide Web, Banff, AB, Canada, 8–12 May 2007; pp. 649–656.
8. Eze, C.S.; Shamir, L. Analysis and prevention of AI-based phishing email attacks. *Electronics* **2024**, *13*, 1839. [CrossRef]
9. Agarwal, K.; Kumar, T. Email spam detection using integrated approach of Naïve Bayes and particle swarm optimization. In Proceedings of the 2018 Second International Conference on Intelligent Computing and Control Systems (ICICCS), Madurai, India, 14–15 June 2018; pp. 685–690.
10. Gokul, S.; PK, N.B. Analysis of phishing detection using logistic regression and random forest. *J. Appl. Inf. Sci.* **2020**, *8*, 7–13.
11. Akinyelu, A.A.; Adewumi, A.O. Classification of phishing email using random forest machine learning technique. *J. Appl. Math.* **2014**, *2014*, 425731.
12. Al-Subaiey, A.; Al-Thani, M.; Alam, N.A.; Antora, K.F.; Khandakar, A.; Zaman, S.A.U. Novel interpretable and robust web-based AI platform for phishing email detection. *Comput. Electr. Eng.* **2024**, *120*, 109625.
13. Rafat, K.F.; Xin, Q.; Javed, A.R.; Jalil, Z.; Ahmad, R.Z. Evading obscure communication from spam emails. *Math. Biosci. Eng* **2022**, *19*, 1926–1943. [PubMed]
14. Bagui, S.; Nandi, D.; Bagui, S.; White, R.J. Machine learning and deep learning for phishing email classification using one-hot encoding. *J. Comput. Sci* **2021**, *17*, 610–623.
15. Verma, P.; Goyal, A.; Gigras, Y. Email phishing: Text classification using natural language processing. *Comput. Sci. Inf. Technol.* **2020**, *1*, 1–12.
16. Noah, N.; Tayachew, A.; Ryan, S.; Das, S. Poster: PhisherCop-An Automated Tool Using ML Classifiers for Phishing Detection. In Proceedings of the 43rd IEEE Symposium on Security and Privacy (IEEE S&P 2022), San Francisco, CA, USA, 23–26 May 2022.
17. Kim, S.; Park, J.; Ahn, H.; Lee, Y. Detection of Korean Phishing Messages Using Biased Discriminant Analysis under Extreme Class Imbalance Problem. *Information* **2024**, *15*, 265. [CrossRef]
18. Alotaibi, R.; Al-Turaiki, I.; Alakeel, F. Mitigating email phishing attacks using convolutional neural networks. In Proceedings of the 2020 3rd International Conference on Computer Applications & Information Security (ICCAIS), Riyadh, Saudi Arabia, 19–21 March 2020; pp. 1–6.

19. Halgaš, L.; Agrafiotis, I.; Nurse, J.R. Catching the Phish: Detecting phishing attacks using recurrent neural networks (RNNs). In Proceedings of the Information Security Applications: 20th International Conference, WISA 2019, Jeju Island, Republic of Korea, 21–24 August 2019; Revised Selected Papers 20; Springer: Berlin/Heidelberg, Germany, 2020, pp. 219–233.
20. Li, Q.; Cheng, M.; Wang, J.; Sun, B. LSTM based phishing detection for big email data. *IEEE Trans. Big Data* **2020**, *8*, 278–288.
21. Devlin, J.; Chang, M.W.; Lee, K.; Toutanova, K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv* **2019**, arXiv:1810.04805.
22. Uddin, M.A.; Sarker, I.H. An Explainable Transformer-based Model for Phishing Email Detection: A Large Language Model Approach. *arXiv* **2024**, arXiv:2402.13871.
23. Gogoi, B.; Ahmed, T. Phishing and Fraudulent Email Detection thrgh Transfer Learning using pretrained transformer models. In Proceedings of the 2022 IEEE 19th India Council International Conference (INDICON), IEEE, Kochi, India, 24–26 November 2022; pp. 1–6.
24. ou Sarker, I.H. Deep learning: A comprehensive overview on techniques, taxonomy, applications and research directions. *SN Comput. Sci.* **2021**, *2*, 420. [CrossRef]
25. Islam, M.R.; Ahmed, M.U.; Barua, S.; Begum, S. A systematic review of explainable artificial intelligence in terms of different application domains and tasks. *Appl. Sci.* **2022**, *12*, 1353. [CrossRef]
26. Doshi, J.; Parmar, K.; Sanghavi, R.; Shekokar, N. A comprehensive dual-layer architecture for phishing and spam email detection. *Comput. Secur.* **2023**, *133*, 103378. [CrossRef]
27. Hoheisel, R.; van Capelleveen, G.; Sarmah, D.K.; Junger, M. The development of phishing during the COVID-19 pandemic: An analysis of over 1100 targeted domains. *Comput. Secur.* **2023**, *128*, 103158. [CrossRef] [PubMed]
28. Alkhalil, Z.; Hewage, C.; Nawaf, L.; Khan, I. Phishing attacks: A recent comprehensive study and a new anatomy. *Front. Comput. Sci.* **2021**, *3*, 563060. [CrossRef]
29. Thakur, K.; Ali, M.L.; Obaidat, M.A.; Kamruzzaman, A. A systematic review on deep-learning-based phishing email detection. *Electronics* **2023**, *12*, 4545. [CrossRef]
30. Liu, Y.; Ott, M.; Goyal, N.; Du, J.; Joshi, M.; Chen, D.; Levy, O.; Lewis, M.; Zettlemoyer, L.; Stoyanov, V. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv* **1907**, arXiv:1907.11692.
31. Chinta, P.C.R.; Moore, C.S.; Karaka, L.M.; Sakuru, M.; Bodepudi, V.; Maka, S.R. Building an Intelligent Phishing Email Detection System Using Machine Learning and Feature Engineering. *Eur. J. Appl. Sci. Eng. Technol.* **2025**, *3*, 41–54. [CrossRef]
32. Meléndez, R.; Ptaszynski, M.; Masui, F. Comparative Investigation of Traditional Machine-Learning Models and Transformer Models for Phishing Email Detection. *Electronics* **2024**, *13*, 4877. [CrossRef]
33. Atawneh, S.; Aljehani, H. Phishing email detection model using deep learning. *Electronics* **2023**, *12*, 4261. [CrossRef]
34. Alhogail, A.; Alsabih, A. Applying machine learning and natural language processing to detect phishing email. *Comput. Secur.* **2021**, *110*, 102414.
35. Fang, Y.; Zhang, C.; Huang, C.; Liu, L.; Yang, Y. Phishing email detection using improved RCNN model with multilevel vectors and attention mechanism. *IEEE Access* **2019**, *7*, 56329–56340.
36. Sakkis, G.; Androutsopoulos, I.; Paliouras, G.; Karkaletsis, V.; Spyropoulos, C.D.; Stamatopoulos, P. A memory-based approach to anti-spam filtering for mailing lists. *Inf. Retr.* **2003**, *6*, 49–73.
37. Klimt, B.; Yang, Y. The enron corpus: A new dataset for email classification research. In *European Conference on Machine Learning*; Springer: Berlin/Heidelberg, Germany, 2004; pp. 217–226.
38. Apache SpamAssassin. Available online: https://zenodo.org/records/8339691 (accessed on 20 September 2024).
39. Craswell, N.; De Vries, A.P.; Soboroff, I. Overview of the TREC 2005 Enterprise Track. In Proceedings of the Fourteenth Text REtrieval Conference, TREC 2005, Gaithersburg, MD, USA, 15–18 November 2005; Volume 5, pp. 1–7.
40. Bratko, A.; Filipic, B.; Zupan, B. Towards Practical PPM Spam Filtering: Experiments for the TREC 2006 Spam Track. In Proceedings of the Fifteenth Text REtrieval Conference, TREC 2006, Gaithersburg, MD, USA, 14–17 November 2006.
41. DeBarr, D.; Wechsler, H. Spam detection using random boost. *Pattern Recognit. Lett.* **2012**, *33*, 1237–1244.
42. Nazario, J. The Online Phishing Corpus. 2023. Available online: http://monkey.org/~jose/wiki/doku.php (accessed on 20 September 2024).
43. Radev, D. CLAIR Collection of Fraud Email. 2008. Available online: https://aclweb.org/aclwiki (accessed on 12 December 2024).
44. Harter, S.P. A probabilistic approach to automatic keyword indexing. Part I. On the distribution of specialty words in a technical literature. *J. Am. Soc. Inf. Sci.* **1975**, *26*, 197–206.
45. Tibshirani, R. Regression shrinkage and selection via the lasso. *J. R. Stat. Soc. Ser. B Stat. Methodol.* **1996**, *58*, 267–288.
46. Shahrivari, V.; Darabi, M.M.; Izadi, M. Phishing detection using machine learning techniques. *arXiv* **2020**, arXiv:2009.11116.
47. Robbins, H.; Monro, S. A stochastic approximation method. *Ann. Math. Stat.* **1951**, *22*, 400–407.
48. Chen, T.; Guestrin, C. XGBoost: A Scalable Tree Boosting System. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, 13–17 August 2016; Volume 1143, pp. 785–794.

49. Odeh, A.; Al-Haija, Q.A.; Aref, A.; Taleb, A.A. Comparative study of catboost, xgboost, and lightgbm for enhanced URL phishing detection: A performance assessment. *J. Internet Serv. Inf. Secur.* **2023**, *13*, 1–11.
50. Salloum, S.; Gaber, T.; Vadera, S.; Shaalan, K. A systematic literature review on phishing email detection using natural language processing techniques. *IEEE Access* **2022**, *10*, 65703–65727.
51. Breiman, L. Random forests. *Mach. Learn.* **2001**, *45*, 5–32.
52. Salloum, S.; Gaber, T.; Vadera, S.; Shaalan, K. Phishing email detection using natural language processing techniques: A literature survey. *Procedia Comput. Sci.* **2021**, *189*, 19–28. [CrossRef]
53. Geurts, P.; Ernst, D.; Wehenkel, L. Extremely randomized trees. *Mach. Learn.* **2006**, *63*, 3–42.
54. Champa, A.I.; Rabbi, M.F.; Zibran, M.F. Curated datasets and feature analysis for phishing email detection with machine learning. In Proceedings of the 2024 IEEE 3rd International Conference on Computing and Machine Intelligence (ICMI), Mt Pleasant, MI, USA, 13–14 April 2024; pp. 1–7.
55. Bai, S.; Kolter, J.Z.; Koltun, V. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv* **2018**, arXiv:1803.01271.
56. Kalchbrenner, N.; Grefenstette, E.; Blunsom, P. A convolutional neural network for modelling sentences. *arXiv* **2014**, arXiv:1404.2188.
57. Wang, P.; Xu, J.; Xu, B.; Liu, C.; Zhang, H.; Wang, F.; Hao, H. Semantic clustering and convolutional neural network for short text categorization. In Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 2: Short Papers), Beijing, China, 26–31 July 2015; pp. 352–357.
58. Zhou, Y.; Li, J.; Chi, J.; Tang, W.; Zheng, Y. Set-CNN: A text convolutional neural network based on semantic extension for short text classification. *Knowl. Based Syst.* **2022**, *257*, 109948. [CrossRef]
59. Moriya, S.; Shibata, C. Transfer learning method for very deep CNN for text classification and methods for its evaluation. In Proceedings of the 2018 IEEE 42nd Annual Computer Software and Applications Conference (COMPSAC), IEEE, Tokyo, Japan, 23–27 July 2018; Volume 2, pp. 153–158.
60. Collobert, R.; Weston, J.; Bottou, L.; Karlen, M.; Kavukcuoglu, K.; Kuksa, P. Natural language processing (almost) from scratch. *J. Mach. Learn. Res.* **2011**, *12*, 2493–2537.
61. Jacovi, A.; Shalom, O.S.; Goldberg, Y. Understanding convolutional neural networks for text classification. *arXiv* **2018**, arXiv:1809.08037.
62. Alzubaidi, L.; Zhang, J.; Humaidi, A.J.; Al-Dujaili, A.; Duan, Y.; Al-Shamma, O.; Santamaría, J.; Fadhel, M.A.; Al-Amidie, M.; Farhan, L. Review of deep learning: Concepts, CNN architectures, challenges, applications, future directions. *J. Big Data* **2021**, *8*, 1–74. [CrossRef]
63. Hochreiter, S. *Long Short-Term Memory*; Neural Computation MIT-Press: Cambridge, MA, USA, 1997.
64. Khataei Maragheh, H.; Gharehchopogh, F.S.; Majidzadeh, K.; Sangar, A.B. A new hybrid based on long short-term memory network with spotted hyena optimization algorithm for multi-label text classification. *Mathematics* **2022**, *10*, 488. [CrossRef]
65. Eang, C.; Lee, S. Improving the Accuracy and Effectiveness of Text Classification Based on the Integration of the Bert Model and a Recurrent Neural Network (RNN_Bert_Based). *Appl. Sci.* **2024**, *14*, 8388. [CrossRef]
66. Hajer, M.A.; Alasadi, M.K.; Obied, A. Transfer Learning Models for E-mail Classification. *J. Cybersecur. Inf. Manag.* **2025**, *15*, 342.
67. Sanh, V.; Debut, L.; Chaumond, J.; Wolf, T. DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv* **2019**, arXiv:1910.01108.
68. Farhangian, F.; Cruz, R.M.; Cavalcanti, G.D. Fake news detection: Taxonomy and comparative study. *Inf. Fusion* **2024**, *103*, 102140.
69. Jamal, S.; Wimmer, H.; Sarker, I.H. An improved transformer-based model for detecting phishing, spam and ham emails: A large language model approach. *Secur. Priv.* **2024**, *7*, e402.
70. Kipf, T.N.; Welling, M. Semi-supervised classification with graph convolutional networks. *arXiv* **2016**, arXiv:1609.02907.
71. Yao, L.; Mao, C.; Luo, Y. Graph convolutional networks for text classification. *Proc. Aaai Conf. Artif. Intell.* **2019**, *33*, 7370–7377.
72. Prasad, R. Phishing Email Detection Using Machine Learning: A Critical Review. In Proceedings of the 2024 IEEE International Conference on Computing, Power and Communication Technologies (IC2PCT), IEEE, Greater Noida, India 9–10 February 2024; Volume 5, pp. 1176–1180.
73. Miltchev, R.; Dimitar, R.; Evgeni, G. Phishing Validation Emails Dataset. Zenodo. 2024. Available online: https://zenodo.org/records/13474746 (accessed on 20 January 2025).
74. Microsoft Defender. 2025. Available online: https://www.microsoft.com/en-us/security/business/microsoft-defender (accessed on 5 March 2025).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.